



MTA

AUDIO EDITOR

Complete MTA (Multi Track Audio) format specification

On-disk structure, audio streams, timing, metadata and validation

Version: 0.2.0 · Revision: 159

Build: 20261006021358

Alessandro De Salvo

braket71@gmail.com

<https://github.com/desalvo/mta-audio-editor>

License: EUPL-1.2

Digital Audio Workstation with MTA support



Windows



macOS



Linux



Android



iOS

Contents

1. Scope and validation level

2. High-level file organization

3. Proprietary media transport

3.1 Transform

3.2 Validation across the corpus

4. EBML/Matroska primitives

5. Audio format

5.1 MP3 frame size

6. SimpleBlock layout

7. Cluster geometry and timebase

8. Cues

9. SeekHead

10. Xing/Info first MP3 frame

10.1 TOC

10.2 CRCs

11. Attachments

12. SYL attachment

12.1 Shared 256-byte keystream

12.2 Common binary prefix

13. COLORS

13.1 Position 127

14. LYRICS and CHORDS

15. MIDITK

16. MtxInfoData XML

17. NoteOn semantics

18. PreCntUsec

19. Reader algorithm

20. Writer algorithm

21. Compatibility levels

22. Validation invariants

23. Known limitations

24. Click and Melody device profiles

MTA (Multi Track Audio) format - consolidated technical specification

Version of this document: 0.2.0-93

1. Scope and validation level

In this documentation **MTA** means **Multi Track Audio**, the multitrack format used to keep musical streams, roles and timed data separately addressable.

This specification documents the on-disk MTA structures validated across the available MTA corpus used by MTA Audio Editor. It distinguishes:

- **Verified byte-level structure:** reproduced from stored bytes and cross-checked on all available stock samples.
- **Verified semantics:** the meaning of a field is supported by its relation to decoded audio/lyrics/MIDI/timeline data.
- **Authoring policy:** how MTA Audio Editor writes a semantically equivalent value where multiple authoring strategies can produce the same valid on-disk result.
- **Hardware acceptance:** separate from format analysis. A generated file can be structurally valid without proving compatibility with every Merish firmware revision.

The verified corpus contains four independent commercial/stock MTA files with 10-14 audio tracks. No copyrighted source media is distributed with the project.

2. High-level file organization

MTA is a Matroska-derived container. The file begins with canonical/readable EBML structures:

```
EBML Header
Segment
  SeekHead
  Void
  Info
  Tracks
  Attachments
  Cues
  Cluster ... stored through proprietary XOR transport
  Cluster ...
  ...
```

The segment data offset is the first byte after the Segment ID and Segment-size VINT. SeekPosition and CueClusterPosition values are relative to this point.

Before the first Cluster, the file is ordinary EBML/Matroska. From the first Cluster byte through EOF, the bytes are stored using the proprietary transport described below.

3. Proprietary media transport

3.1 Transform

The transport is a fixed repeating XOR stream with period 984 bytes:

```
plain[i] = stored[i] XOR key[i mod 984]
stored[i] = plain[i] XOR key[i mod 984]
```

where $i = 0$ at the first byte of the first Cluster.

The key phase:

- starts exactly at the first Cluster ID byte;
- continues through EOF;
- does not reset at Cluster boundaries;
- does not reset at Cue boundaries;
- is symmetric for encoding and decoding.

The implementation stores this documented key as the MEDIA_XOR_KEY compatibility constant.

Integrity identity:

```
period: 984 bytes
SHA-256:
bcb30443707bdc8b651c1a58aa4152438ce5a6632cc98b294501eb08adbb3547
```

3.2 Validation across the corpus

The 984-byte period is consistently verifiable from predictable canonical Matroska plaintext across the corpus:

- Cluster element ID 1F 43 B6 75;
- Timecode element;
- SimpleBlock element IDs and EBML sizes;
- track-number VINTs;
- relative block timecodes;
- block flags;
- MPEG audio sync/header bytes.

Applying the documented period from the first Cluster to EOF turns every tested file into canonical Matroska readable by FFmpeg through the complete media region.

4. EBML/Matroska primitives

MTA uses ordinary EBML IDs and minimal-width EBML size VINTs in the verified regions.

Important IDs:

Element	Hex ID
EBML	1A45DFA3
Segment	18538067
SeekHead	114D9B74
Info	1549A966
Tracks	1654AE6B
Attachments	1941A469
Cues	1C53BB6B
Cluster	1F43B675
Timecode	E7
SimpleBlock	A3
CuePoint	BB
CueTime	B3
CueTrackPositions	B7
CueClusterPosition	F1

Segment size covers exactly the remainder of the file to EOF in the verified samples.

5. Audio format

Audio streams are:

```
MPEG-1 Layer III
sample rate: 44100 Hz
channels: stereo
nominal bit rate: 320000 bit/s
```

Each Matroska SimpleBlock carries exactly one MP3 frame.

5.1 MP3 frame size

For 320 kbit/s MPEG-1 Layer III at 44.1 kHz:

```
base = floor(144 * 320000 / 44100) = 1044 bytes
```

Frames therefore have 1044 or 1045 bytes according to the padding bit.

Packet 0 is a 1044-byte, unpadded Info frame.

The later padding schedule is exactly reproduced by:

```
padding = [0]    # packet zero, Info frame
state = 4

for each later musical frame:
    state += 44
    if state >= 49:
        padding.append(1)
        state -= 49
    else:
        padding.append(0)
```

Frame size is $1044 + \text{padding}[n]$.

6. SimpleBlock layout

A stock audio SimpleBlock has seven bytes of Matroska overhead before the MP3 frame:

```
A3
<size VINT, normally 2 bytes>
<track number VINT, 1 byte: 81..8E in 14-track examples>
<relative timecode, signed big-endian int16>
80      # block flags
<MP3 frame>
```

Thus a block is normally 1051 or 1052 bytes.

Blocks are ordered **frame-major**, not track-major:

```
frame time 0: track 1, track 2, ... track N
frame time 1: track 1, track 2, ... track N
...
```

7. Cluster geometry and timebase

A complete Cluster contains ten MP3 frame-times for every audio track.

Relative SimpleBlock timecodes are:

```
0
144
288
432
576
720
864
1008
1152
1296
```

Verified Matroska timing values:

```
TimecodeScale    = 181392 ns
DefaultDuration  = 26122448 ns
```

The absolute Cluster timecode is exactly reproduced by:

```
cluster_timecode(i) = floor(i * (800000000 / 555513) + 1/16)
```

where *i* is the zero-based Cluster index.

The final Cluster can contain fewer than ten frame-times. It uses the minimal EBML size VINT needed for its actual payload. This detail explains the earlier one-byte discrepancy observed at the end of the Katrina sample.

8. Cues

A Cue is emitted every four Clusters, therefore every 40 MP3 packets per track.

```
cue_count = ceil(packet_count_per_track / 40)
Cue n -> Cluster 4*n
```

CueClusterPosition is relative to Segment data start.

Cue times and positions can be generated deterministically from the packet count, frame-size schedule, track count and exact Cluster-size model.

For the verified field widths, a CuePoint has total size:

```
11
+ byte_length(CueTime)
+ byte_length(CueClusterPosition)
```

9. SeekHead

The SeekHead references at least the major pre-media structures and the first Cluster.

For the verified value widths, one Seek entry consists of:

```
Seek
  SeekID
  SeekPosition
```

The SeekID part is fixed-width for a known target. With an *n*-byte minimal unsigned SeekPosition, the observed entry size follows:

```
13 + n bytes
```

Seek positions are Segment-relative.

Because the encoded width of positions can change when offsets cross a byte boundary, a writer should solve the pre-media layout using fixed-point iteration:

1. assume widths;
2. serialize or calculate all element lengths;
3. recompute positions;
4. recompute minimal widths;

5. repeat until widths and offsets stop changing.

10. Xing/Info first MP3 frame

The first packet of every track is an unpaddinged 1044-byte MP3 Info frame.

Observed/verified layout:

Offset	Field
0..3	MPEG audio header
36..39	ASCII Info
40..43	flags 0x0000000f
44..47	musical frame count, Info excluded
48..51	total MP3 bytes, Info included
52..151	Xing TOC, 100 bytes
152..155	quality
156..	encoder string, Lavc58.13 in stock corpus
177..179	packed 12-bit encoder delay + 12-bit padding
184..187	music length
188..189	music CRC
190..191	tag CRC

Relations:

```
Info.frames = total_packets - 1
Info.bytes  = sum(all MP3 frames, including Info)
music_length = Info.bytes
```

10.1 TOC

The 100-byte TOC matches the FFmpeg/LAME-style 400-entry sampling-bag algorithm.

Conceptually:

```

bag = [0] * 400
want = 1
seen = 0
pos = 0

for each musical frame:
    total_size += frame_size
    seen += 1

    if seen == want:
        bag[pos] = total_size
        pos += 1
        seen = 0

    if pos == 400:
        bag[:200] = bag[1:400:2]
        want *= 2
        pos = 200

toc[0] = 0
for i in range(1, 100):
    j = i * pos // 100
    toc[i] = min(256 * bag[j] // total_size, 255)

```

10.2 CRCs

Both CRC fields use reflected CRC-16/ANSI with polynomial 0xA001 and initial value zero.

Music CRC:

```

CRC over every musical MP3 frame in order
Info frame excluded

```

Tag CRC:

```

CRC over Info-frame bytes 0..189

```

The music CRC is inserted before the tag CRC is calculated.

11. Attachments

Verified stock files contain four attachments:

```

original SYL
modified SYL
original MtxInfoData XML
modified MtxInfoData XML

```

In the inspected corpus the original and modified copies are byte-identical.

MTA Audio Editor preserves imported attachments byte-for-byte whenever possible. If FFmpeg attachment extraction fails, the MTA compatibility parser can recover Matroska FileData conservatively using sizes obtained from the parsed container.

12. SYL attachment

The SYL file is a complete ID3v2.3 tag plus M-Live proprietary sections.

Observed sections:

```
DELTA
LYRICS
COLORS
MIDITK
CHORDS
```

Standard ID3 frames can contain title, artist, BPM, key, duration, publisher, composer, genre, copyright, year, album and ISRC.

12.1 Shared 256-byte keystream

LYRICS, COLORS, MIDITK and CHORDS use one shared 256-byte XOR keystream.

COLORS exposes the entire sequence because byte zero of its first 256 normal records directly reveals the active key byte after cancellation of the record's known LF and global XOR values.

Across all verified stock samples the validated 256-byte sequence is identical and is a permutation of all byte values 00..FF.

12.2 Common binary prefix

Verified proprietary section bodies begin with:

```
28 B6 A9
```

before the encrypted section payload.

13. COLORS

Normal physical COLORS records are 15 bytes.

After removing the transport transform, the verified plaintext layout is:

```
LF d d RS d d 'm' p p p '=' ':' 'k' NUL next_minute
```

where decimal digit bytes store:

```
centiseconds = d1*1000 + d2*100 + d3*10 + d4
position      = p1*100 + p2*10 + p3
time_ms       = current_minute*60000 + centiseconds*10
```

Encryption phase:

```
key_index = (record_index - 17*byte_index) mod 256
stored = plain XOR key[key_index] XOR 0x0A
```

The last physical record is a distinct terminator and is not a normal event.

13.1 Position 127

The value 127 is a **display-page reset control**, not a character position.

Across the verified corpus it:

- follows the final progressive highlight of the previous displayed lyric line;
- occurs immediately before the first lyric event of the next display page;
- is not used as an ordinary progressive index.

A writer should emit it when its own lyric display/page-layout engine starts a new displayed page.

14. LYRICS and CHORDS

After the three-byte prefix, these sections use a flat byte-stream phase:

```
key_index = 239 * flat_payload_offset mod 256
stored = plain XOR key[key_index] XOR 0x0A
```

Text/chord bytes use one additional transform:

```
text_byte ^= 0x30
```

Records terminate with the verified marker:

```
'= : k' NUL next_minute
```

(with the literal stored/plain byte sequence =:k\0X after decoding).

Timing uses the same minute-block plus four-digit centisecond scheme as COLORS.

15. MIDITK

MIDITK uses:

```
key_index = 239 * (payload_offset - 3) mod 256
stored = midi_byte XOR key[key_index] XOR 0x0A XOR 0x30
```

Decoding reconstructs a byte-valid Standard MIDI File.

Verified MIDI characteristics:

```
SMF format 0
one MTrk
PPQ 480
tempo meta-event
marker meta-events
end-of-track meta-event
```

Markers include musical sections such as Intro, Verse, Chorus, Bridge, Coda and Fine.

16. MtxInfoData XML

The XML attachment has root MtxInfoData.

Verified logical groups include:

```
Identity
General
TrkAudio / Track
```

Observed semantics include:

- user/application identity fields;
- base key;
- base BPM;
- duration in seconds;
- pre-count in microseconds;
- track number/name/type;
- routing;
- volume;
- mute/solo;
- NoteOn.

Some stock XML payloads contain terminal zero padding. The padding is part of the attachment bytes and is preserved on import.

17. NoteOn semantics

NoteOn is a semicolon-separated 0/1 vector representing approximately one activity decision per second for a source track.

Corpus analysis:

```
4 songs
50 tracks
12,654 one-second bins
36/50 tracks exactly reproduced by a best single decoded-RMS threshold
45/50 differ by at most one bin
21/12654 total best-threshold disagreements = 0.166%
```

The residual disagreements cluster around:

- source onsets;
- source releases;
- codec overlap/leakage;
- very low-level sustained material.

This means NoteOn is a **pre-encode/source activity mask**, not a value calculated from a fixed threshold on the final decoded MP3.

The final encoded media does not uniquely determine the source-side activity detector; several detectors can produce the same resulting activity mask.

Authoring policy:

- imported MTA: preserve original NoteOn exactly;

- newly authored project: derive one-second activity from the editor's pre-encode clip/source timeline.

That policy preserves the documented semantics without imposing an artificial universal MP3 threshold.

18. PreCntUSec

PreCntUSec is an explicit pre-count/lead-in timeline duration in microseconds.

It is related to musical lead-in but is not consistently equal to a simple four-beat formula computed from rounded BaseBpm.

A writer should therefore treat it as an explicit timeline property.

19. Reader algorithm

A robust reader should:

1. parse the readable EBML pre-media area;
2. locate the first Cluster via SeekHead/Cues;
3. inspect the bytes at that offset;
4. if canonical 1F43B675 is present, treat media as ordinary Matroska;
5. otherwise XOR-decode a probe using the 984-byte key;
6. if that yields 1F43B675, decode the entire media region to a temporary canonical Matroska view;
7. use FFmpeg/FFprobe against the canonical view for media extraction;
8. separately preserve/analyze original attachment bytes;
9. parse SYL/XML only when their structural invariants validate;
10. retain unknown bytes for lossless round-trip safety.

20. Writer algorithm

For the MTA media transport layer, a writer can:

1. render each output slot;
2. encode/mux a canonical Matroska representation;
3. preserve or create required attachments;
4. locate the first canonical Cluster;
5. XOR-transform every byte from that Cluster to EOF using the 984-byte stream;
6. leave all pre-media bytes untouched;
7. verify that reapplying the transform restores a canonical Cluster;
8. optionally deobfuscate to a temporary file and run FFprobe as a structural acceptance test.

For a stock-layout deterministic writer, additionally use the exact packet, Cluster, Cue, SeekHead and Xing/Info formulas in this document instead of relying on a generic Matroska muxer.

21. Compatibility levels

MTA Audio Editor distinguishes these compatibility levels:

Level	Meaning
Structural read	Container can be parsed and streams/attachments recovered
Extended read	XOR media, SYL sections and MtxInfoData semantics decoded
Structural write	Generated output has valid Matroska plus proprietary XOR transport
Metadata-preserving write	Imported proprietary attachments are preserved byte-for-byte
Native metadata authoring	New SYL/XML metadata generated according to documented semantics
Hardware acceptance	File successfully tested on a specific M-Live/Merish device/firmware

The first four levels are testable in CI. Hardware acceptance requires physical target equipment or a trusted vendor implementation.

22. Validation invariants

Automated validation should check:

```
EBML header present
Segment present
first Cluster discoverable
raw first Cluster matches the stored MTA transport form
XOR-decoded first Cluster begins 1F43B675
XOR round-trip is byte-identical
deobfuscated media is FFprobe-readable
stream count equals expected output slots
attachments remain extractable
project import/export does not mutate preserved original attachment bytes
```

For stock-layout writer tests, additionally verify:

```
MP3 frame sizes
padding accumulator
SimpleBlock overhead
frame-major order
Cluster timecodes
Cluster spans
Cue count/times/positions
Info.frames
Info.bytes
TOC
music CRC
tag CRC
Segment size
EOF alignment
```

23. Known limitations

The on-disk structures required to read and write the verified format are documented. The remaining uncertainty is not a binary parsing problem but device/vendor acceptance and authoring choices for which multiple valid implementations can lead to equivalent lossy final media.

Do not claim bit-identical reproduction of every vendor encoder version unless compared against a controlled reference generated by that exact version.

24. Click and Melody device profiles

MTA Audio Editor keeps Click/Metronome and Melody roles separate from ordinary musical stems and applies a device profile at export time.

Profile	Click	Melody
Merish5 / Xynthia2 MTA8	8	7
B.Beat / B.Beat Plus / Evo / Pro16 / DIVO MTA8	7	8
M-Live MTA16 corpus default	1	9

For MTA16, the default is derived from the available files: Metronome is stream 1 in every observed sample; an explicit Melody Track appears most often in stream 9 and in one sample in stream 10. Stream 9 is therefore a practical default rather than a mandatory rule. The user can change the mapping before export.

When a device profile requires a physical stream number that would otherwise be missing, the writer inserts silent intermediate streams so the Click or Melody role remains at the requested physical position in the resulting MTA file.